

kANNolo: Sweet and Smooth Approximate k -Nearest Neighbors Search

Leonardo Delfino⁴, Domenico Erriquez⁴, Silvio Martinico^{3,1},
Franco Maria Nardini^{1[0000-0003-3183-334X]},
Cosimo Rulli^{1[0000-0003-0194-361X]}, and
Rossano Venturini^{2,1[0000-0002-9830-3936]}

¹ ISTI-CNR, Pisa, Italy, {name.surname}@isti.cnr.it

² University of Pisa, Italy, rossano.venturini@unipi.it

³ University of Pisa, Italy, silvio.martinico@phd.unipi.it

⁴ University of Pisa, Italy, {l.delfino1, d.erriquez1}@studenti.unipi.it

Abstract. Approximate Nearest Neighbors (ANN) search is a crucial task in several applications like recommender systems and information retrieval. Current state-of-the-art ANN libraries, although being performance-oriented, often lack modularity and ease of use. This translates into them not being fully suitable for easy prototyping and testing of research ideas, an important feature to enable. We address these limitations by introducing **kANNolo**, a novel—research-oriented—ANN library written in Rust and explicitly designed to combine usability with performance effectively. **kANNolo** is the first ANN library that supports dense and sparse vector representations made available on top of different similarity measures, e.g., euclidean distance and inner product. Moreover, it also supports vector quantization techniques, e.g., Product Quantization, on top of the indexing strategies implemented. These functionalities are managed through Rust traits, allowing shared behaviors to be handled abstractly. This abstraction ensures flexibility and facilitates an easy integration of new components. In this work, we detail the architecture of **kANNolo** and demonstrate that its flexibility does not compromise performance. The experimental analysis shows that **kANNolo** achieves state-of-the-art performance in terms of speed-accuracy trade-off while allowing fast and easy prototyping, thus making **kANNolo** a valuable tool for advancing ANN research. Source code available on GitHub: <https://github.com/TusKANNy/kannolo>.

Keywords: Nearest Neighbors Search · Software library · Rust.

1 Introduction

Nearest Neighbor (NN) search is a fundamental problem in many domains of computer science, including image processing, information retrieval, and recommendation systems. NN algorithms search through a dataset X of d -dimensional vectors to identify the k nearest neighbors to a given query point $x \in \mathbb{R}^d$. While exact solutions to this problem are computationally feasible in low-dimensional

spaces, exact nearest neighbors search algorithms are no better than brute-force linear scans in high-dimensional settings. Approximate Nearest Neighbors (ANN) techniques tackle this aspect by giving up on exactness to offer a valuable trade-off between accuracy and efficiency. These techniques are especially popular nowadays due to the advent of effective learned vector representations (embeddings) powered by Large Language Models. In response to this trend, several well-known ANN libraries have been developed.

Many of these libraries, such as NMSlib [2], DiskANN [16] (Microsoft), and ScaNN [7] [17] (Google) implement highly efficient ANN search algorithms optimized for speed and scalability. However, these solutions are typically built around specific indexing or quantization techniques, with rigid codebases that do not support easy integration of new components or generalization beyond their original design. This lack of flexibility poses a significant limitation for researchers and developers who need to prototype or modify ANN algorithms. Even FAISS [4], Meta’s comprehensive library for ANN search which provides a wide variety of indexing techniques and quantization methods, lacks the modularity required for advancing research in the field. FAISS is heavily engineered for performance, leading to specialized implementations of each ANN/quantization technique. For example, in the case of the IVF (Inverted File Index), FAISS provides ten distinct classes that implement several versions of IVF index coupled with different quantizers. This highly specific architecture makes it challenging to introduce new functionalities that apply across different IVF variants, limiting flexibility in experimental settings. Moreover, FAISS suffers from a lack of support for sparse datasets and —like other more specialized libraries— it lacks comprehensive and easily accessible documentation. This can make it challenging to quickly understand the codebase and the functioning of the library.

In this work, we introduce **kANNolo**, a new Rust library for ANN. Unlike existing libraries, **kANNolo** is designed having ANN researchers in mind. We design it by prioritizing the ease of modification and prototyping while also supporting generality. Indeed—and differently from available competitors—**kANNolo** implements state-of-the-art indexing techniques for both dense and sparse vectors, as well as quantization techniques. Specifically, we design **kANNolo** as a collection of abstract key components such as: 1) the dataset representation (dense/sparse), 2) the quantization method, and 3) the distance/similarity measure. Second, we exploit Rust traits—a mechanism for specifying shared behavior across types, defining a set of methods that implementing types must provide—to implement the design above in a way that fully matches the potential unlocked by the abstraction. This modular design allows users to easily develop and integrate new indexes/quantizers and test them with minimal effort so to greatly simplify prototyping and experimentation in ANN research. Although being designed for research, **kANNolo** does not give up on performance. Through comprehensive benchmarks on two publicly available datasets, SIFT1M and MS MARCO, we show that **kANNolo** performs on par or better with the state-of-the-art competitors on both dense and sparse data, with a peak to $11.1\times$ and $2.1\times$ speedups

over the best competitors respectively on dense and sparse data, establishing itself as one of the most competitive ANN search libraries.

2 Design of kANNolo

kANNolo is built with a modular architecture, consisting of four main components:

- **One-Dimensional Arrays:** The `DArray1` trait provides a unified interface for dense and sparse vectors. This abstraction allows us to build indexes and perform ANN search independently on the kind of vectors we use.
- **Quantizer:** Quantizers transform high-dimensional data into more compact representations. They are abstracted through a `Quantizer` trait, enabling flexibility in the type of quantization used. kANNolo also supports an *Identity quantizer* which leaves the data unchanged, providing a standard interface regardless of whether any vector quantization method is applied.
- **Query Evaluator:** The `QueryEvaluator` trait is responsible for computing distances (or similarities) between dataset items and query points, and for ranking the results accordingly. It is tightly integrated with the quantizer, ensuring that all distance computations are conducted through a unified interface. This design abstracts the complexity of handling different data representations or quantization methods.
- **Dataset:** The `Dataset` trait represents a collection of one-dimensional arrays equipped with a quantizer. It dynamically integrates with a query evaluator during search operations and provides access to the data by masking the details on the kind of vectors it contains.

To better understand how these components interact, we illustrated the indexing pipeline of kANNolo in Figure 1 and the retrieval process in Figure 2.

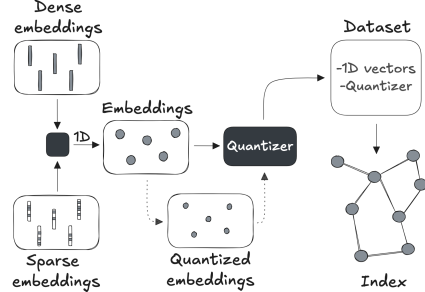


Fig. 1: Indexing with kANNolo.

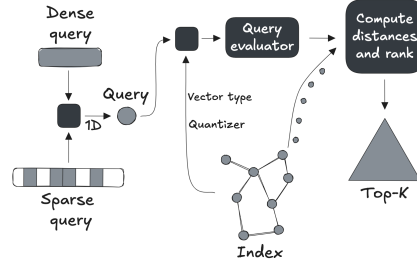


Fig. 2: Retrieval with kANNolo.

In kANNolo, the dataset provides access to the data while the index is meant to pinpoint which vector shall be compared with the query. The indexing method currently implemented in kANNolo is the Hierarchical Navigable Small World graph (HNSW) [11], a graph index with a hierarchical links structure that has proved to be a state-of-the-art method both for dense and sparse retrieval. The quantization method that kANNolo implements is the Product Quantization [8], which quantizes the data into subspaces independently.

2.1 Demo at ECIR 2025

Our demonstration at ECIR 2025 will focus on how easy is developing code on top of **kANNolo**. We will organize it as a live coding session. Attendees will help us write the code for indexing and searching a dataset.

3 Experiments

In this section, we evaluate the performance of **kANNolo** by comparing it to the best ANN libraries implementing the same methods.

Datasets. For the dense domain, we evaluate **kANNolo** with Euclidean distance on the widely-known SIFT1M dataset [10], consisting of vectors of handcrafted visual descriptors of images. Additionally, we test **kANNolo** with inner product on two sets of embeddings from the MS MARCO passages dataset [13], produced by the STAR [18] and DRAGON [9] models. For sparse retrieval, we use inner product on a sparse version of the MS MARCO dataset, where the SPLADE model generates embeddings [5]. For additional details about the tested datasets refer to Table 1.

Table 1: Summary of the datasets employed in our experimental evaluation.

Dataset Name	Data Type	Measure	Cardinality	Dimensionality
SIFT1M	Dense	L2	1,000,000	128
DRAGON/STAR	Dense	IP	8,841,823	768
SPLADE	Sparse	IP	8,841,823	-

Competitors. Dense competitors were selected based on their performance on the wide-recognized benchmark repository ANN benchmarks [1]. Specifically, they are: 1) FAISS, Meta’s library for ANN search with support for a wide range of ANN methods, including HNSW and PQ; 2) HNSWLIB [11], the original implementation of the HNSW algorithm, part of the nmslib library [2]; 3) N2 [12], a lightweight ANN library implementing HNSW.

The sparse competitors are the winner submissions of the “Sparse Track” at NeurIPS 2023 Big ANN challenge [15], namely GRASSRMA [6] and PYANN [14]. These methods implement HNSW with only some minor optimizations, as **kANNolo**. We leave the comparison with inverted indexes for future work [3].

Reproducibility and Hardware Details. We implemented **kANNolo** in Rust (version 1.83) with the compiler’s highest optimization level enabled. We conduct experiments on a server with one Intel i9-9900K CPU clocked at 3.60GHz, with 64 GiB of RAM. We query the indexes using a single thread.

3.1 Experimental Results

In this section, we experimentally show that **kANNolo** meets both its goals:

Modularity. The tests on both sparse and dense datasets, with plain and product quantizers, were conducted using the same indexing algorithm, which operates independently of the vector representation, quantizer, and similarity measure used. On the other hand, all our competitors have specialized implementations tailored for the vector type or quantization method used;

Performance: kANNolo achieves an improvement over state-of-the-art libraries in terms of accuracy-speed trade-off (Figure 3). On dense data, kANNolo is competitive with the state-of-the-art libraries on SIFT1M dataset, while it outperforms its competitors on MS MARCO. In particular, kANNolo is up to $11.1\times$ faster than FAISS as it builds the HNSW graph on the original representations of the vectors rather than on the quantized ones. On sparse vectors kANNolo outperforms both competitors, with up to $2.1\times$ speedup.

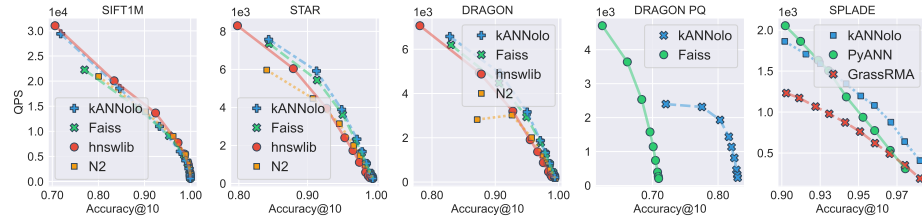


Fig. 3: Accuracy vs. queries per second. Left to right: Dense vectors, no quantization (1-3), dense vectors with product quantization (4), sparse vectors, no quantization (5).

4 Conclusions and Future Work

We introduced kANNolo, a Rust-based library for approximate nearest neighbors search. It is designed to ease the prototyping and development of new ANN algorithms. Our extensive benchmarking on several public dense/sparse datasets shows that kANNolo achieves genuine modularity while also delivering state-of-the-art performance against well-known competitors. These results confirm that kANNolo is a valuable tool for effectively advancing ANN research.

As future work, we plan to extend kANNolo with additional indexing and quantization methods, thereby providing researchers with a broader experimental playground.

Acknowledgments

This work was partially supported by the Horizon Europe RIA “Extreme Food Risk Analytics” (EFRA), grant agreement n. 101093026, by the PNRR - M4C2 - Investimento 1.3, Partenariato Esteso PE00000013 - “FAIR - Future Artificial Intelligence Research” - Spoke 1 “Human-centered AI” funded by the European Commission under the NextGeneration EU program, by the PNRR ECS00000017 Tuscany Health Ecosystem Spoke 6 “Precision medicine & personalized healthcare” funded by the European Commission under the NextGeneration EU programme, by the MUR-PRIN 2017 “Algorithms, Data Structures and Combinatorics for Machine Learning”, grant agreement n. 2017K7XPAN_003, and by the MUR-PRIN 2022 “Algorithmic Problems and Machine Learning”, grant agreement n. 20229BCXNW.

Disclosure of Interests The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Aumüller, M., Bernhardsson, E., Faithfull, A.: Ann-benchmarks: A benchmarking tool for approximate nearest neighbor algorithms. *Information Systems* **87** (02 2019), <https://github.com/erikbern/ann-benchmarks>
2. Boytsov, L., Naidan, B.: Engineering efficient and effective non-metric space library. In: Brisaboa, N.R., Pedreira, O., Zezula, P. (eds.) *Similarity Search and Applications - 6th International Conference, SISAP 2013, A Coruña, Spain, October 2-4, 2013, Proceedings. Lecture Notes in Computer Science*, vol. 8199, pp. 280–293. Springer (2013). https://doi.org/10.1007/978-3-642-41062-8_28, https://doi.org/10.1007/978-3-642-41062-8_28
3. Bruch, S., Nardini, F.M., Rulli, C., Venturini, R.: Efficient inverted indexes for approximate retrieval over learned sparse representations. In: *Annual International ACM SIGIR Conference on Research and Development in Information Retrieval* (2024), <https://api.semanticscholar.org/CorpusID:269449081>
4. Douze, M., Guzhva, A., Deng, C., Johnson, J., Szilvasy, G., Mazaré, P.E., Lomeli, M., Hosseini, L., Jégou, H.: The faiss library. *arXiv preprint arXiv:2401.08281* (2024)
5. Formal, T., Piwowarski, B., Clinchant, S.: Splade: Sparse lexical and expansion model for first stage ranking. In: *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*. p. 2288–2292. SIGIR '21, Association for Computing Machinery, New York, NY, USA (2021). <https://doi.org/10.1145/3404835.3463098>, <https://doi.org/10.1145/3404835.3463098>
6. GrassRMA. <https://github.com/Leslie-Chung/GrassRMA>
7. Guo, R., Sun, P., Lindgren, E., Geng, Q., Simcha, D., Chern, F., Kumar, S.: Accelerating large-scale inference with anisotropic vector quantization. In: *International Conference on Machine Learning* (2020), <https://arxiv.org/abs/1908.10396>
8. Jégou, H., Douze, M., Schmid, C.: Product quantization for nearest neighbor search. *IEEE transactions on pattern analysis and machine intelligence* **33**, 117–28 (01 2011). <https://doi.org/10.1109/TPAMI.2010.57>
9. Lin, S., Asai, A., Li, M., Oguz, B., Lin, J., Mehdad, Y., Yih, W., Chen, X.: How to train your DRAGON: diverse augmentation towards generalizable dense retrieval. *CoRR abs/2302.07452* (2023). <https://doi.org/10.48550/ARXIV.2302.07452>, <https://doi.org/10.48550/arXiv.2302.07452>
10. Lowe, D.: Object recognition from local scale-invariant features. In: *Proceedings of the Seventh IEEE International Conference on Computer Vision*. vol. 2, pp. 1150–1157 vol.2 (1999). <https://doi.org/10.1109/ICCV.1999.790410>
11. Malkov, Y.A., Yashunin, D.A.: Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Trans. Pattern Anal. Mach. Intell.* **42**(4), 824–836 (apr 2020). <https://doi.org/10.1109/TPAMI.2018.2889473>, <https://github.com/nmslib/hnswlib>
12. N2. <https://github.com/kakao/n2>
13. Nguyen, T., Rosenberg, M., Song, X., Gao, J., Tiwary, S., Majumder, R., Deng, L.: MS MARCO: A human generated machine reading comprehension dataset. In: Besold, T.R., Bordes, A., d’Avila Garcez, A.S., Wayne, G. (eds.) *Proceedings of the Workshop on Cognitive Computation: Integrating neural and symbolic approaches 2016 co-located with the 30th Annual Conference on Neural Information Processing Systems (NIPS 2016)*, Barcelona, Spain, December 9, 2016. *CEUR Workshop Proceedings*, vol. 1773. CEUR-WS.org (2016), https://ceur-ws.org/Vol-1773/CoCoNIPS_2016_paper9.pdf

14. pyanns. <https://github.com/veaaaab/pyanns>
15. Simhadri, H.V., Aumüller, M., Ingber, A., Douze, M., Williams, G., Manohar, M.D., Baranchuk, D., Liberty, E., Liu, F., Landrum, B., Karjika, M., Dhulipala, L., Chen, M., Chen, Y., Ma, R., Zhang, K., Cai, Y., Shi, J., Chen, Y., Zheng, W., Wan, Z., Yin, J., Huang, B.: Results of the big ann: Neurips'23 competition (2024), <https://big-ann-benchmarks.com/neurips23.html>
16. Simhadri, H.V., Krishnaswamy, R., Srinivasa, G., Subramanya, S.J., Antonijević, A., Pryce, D., Kaczynski, D., Williams, S., Gollapudi, S., Sivashankar, V., Karia, N., Singh, A., Jaiswal, S., Mahapatro, N., Adams, P., Tower, B., Patel, Y.: DiskANN: Graph-structured Indices for Scalable, Fast, Fresh and Filtered Approximate Nearest Neighbor Search (2023), <https://github.com/Microsoft/DiskANN>
17. Sun, P., Simcha, D., Dopson, D., Guo, R., Kumar, S.: Soar: Improved indexing for approximate nearest neighbor search. In: Neural Information Processing Systems (2023), <https://arxiv.org/abs/2404.00774>
18. Zhan, J., Mao, J., Liu, Y., Guo, J., Zhang, M., Ma, S.: Optimizing dense retrieval model training with hard negatives. In: Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval. p. 1503–1512. SIGIR '21, Association for Computing Machinery, New York, NY, USA (2021). <https://doi.org/10.1145/3404835.3462880>, <https://doi.org/10.1145/3404835.3462880>